VitalFlow V1 — Charge capture workflow

How the clinician or biller reviews, captures, and finalizes charge lines from an encounter — turning a visit into billable work ready for claim generation. This doc is the UI + service companion to docs/billing-rcm.md §3.1.

See also:

- docs/billing-rcm.md — data model + state machine
- docs/encounter-workspace.md — vertical-card stack the capture card lives in
- docs/permissions-matrix.md — `billing:*` + new `charges:capture`

---

1. Workflow

The card sits in the encounter workspace between the AI review card and Documents. It reads + writes against `public.charges` (ChargeLine table) and the encounter's `diagnosis_assignments`.

```
Encounter page
├── Clinical note
├── AI review
├── Charge capture    ← this slice
│      ├── table of lines (status-aware)
│      ├── [ Add line ]
│      ├── [ Post all drafts ]
│      └── "Ready for claim" banner when appropriate
├── Documents
└── Version history
```

Happy path:

1. Clinician or biller opens the encounter.
2. `getChargeCaptureContext` fetches current charges + encounter diagnoses + permissions.
3. User adds one or more draft lines (CPT/HCPCS, units, price, DX).
4. User clicks `Post all drafts` → transaction promotes all valid drafts to `posted`.
5. Card rerenders with "Ready for claim" banner when rollup is `ready_for_claim`.
6. User follows the link to Create Claim (separate slice).

---

2. Status model

Per ChargeLine (unchanged from 5A): `draft → posted → billed → voided`.

Aggregate rollup computed in the service layer:

Lines in aggregate	`rollupStatus`	UI banner
None	`empty`	"No charges captured"
Any `draft`	`draft`	"Review and post"
All `posted`, none `billed`	`ready_for_claim`	"Ready for claim" + link
Any `billed`	`on_claim`	"On submitted claim"
All `voided`	`voided`	"All charges voided"

Only `draft → posted` and `posted → voided` are driven from this UI. `posted → billed` happens as a side effect of `ClaimService.createFromCharges`.

---

3. UI design

Single top-level `<ChargeCaptureCard>` server component. States A (no perm / hidden), B (empty), C (lines present), D (terminal).

```
apps/web/src/app/(app)/encounters/[id]/charge-capture/
├── ChargeCaptureCard.tsx
├── ChargeLineTable.tsx
├── AddChargeLineForm.tsx
├── DiagnosisPicker.tsx
├── shared.tsx                  ChargeStatusBadge, RollupBanner
├── actions.ts                  5 server actions
└── getChargeCaptureContext.ts
```

No `"use client"`. Add/edit/delete are POST-redirect-GET server actions. Diagnosis picker uses multi-checkbox + free-text input (no combobox JS).

---

4. Validation rules

Rule	Where	When
CPT OR HCPCS required, not both	Zod + service	create, update
`units > 0`, `unit_price_minor >= 0`	Zod + DB	create, update
`service_date <= today`	service	create, update, post

`icd10_codes.length >= 1`	service	**post only**
ICD-10 format `^[A-Z][0-9]{2}(\.[0-9A-Z]{1,4})?$`	Zod	create, update
Status != draft → no update / no delete	service	update, delete
Status != draft → no post	service	post
Line on submitted claim → no void	service (join to claims)	void
ICD-10 codes not in encounter DX list	service (WARN only)	post
Duplicate CPT+modifier same service date	service (WARN only)	post
`posted_by = session.userId`	service	post
Impersonated user cannot post	session check	post

Warnings show as amber banners; errors as red banners with `?error=` query string.

---

5. Permissions

Adds one new permission: `charges:capture`.

Capability	Who
View the card	`billing:read` (billers, owners) OR `charges:capture` (providers)
Add / update / delete draft lines	`charges:capture` or `billing:write`
Post drafts	`charges:capture` (own lines) or `billing:write` (any)
Void posted lines	`billing:write` only

Role grants (updated in `rbac.ts`):

Role	billing:read	billing:write	charges:capture
`practice_owner`	✓	✓	✓
`office_admin`	✓	✓	✓
`biller`	✓	✓	✓
`physician`			✓
`nurse_ma`			✓
`scheduler`			

Clean separation of duties: providers capture their own visits; only billers void.

---

6. Edge cases

- **Encounter with no diagnoses** — picker shows empty state; user must type at least one ICD-10 before post.
- **AI-accepted diagnoses** — already in `diagnosis_assignments`, show up in the picker automatically.
- **Encounter cancelled** — card reads as read-only; post disabled; void still works.
- **Only draft line voided** — rollup = `voided`; Post button hides.
- **Dup CPT+modifier same day** — allowed, amber warning on post.
- `service_date` ≠ **encounter date** — allowed (labs, procedures run next day).
- **Concurrent edits** — last write wins at row level. V1 accepts this; add optimistic version check if it bites.
- **Impersonation** — post refused at server-action layer.
- **Already-billed encounter** — adding new draft lines permitted; they're a separate future claim, not an amendment.

---

7. Acceptance criteria

- [ ] Card hidden when user lacks both `billing:read` and `charges:capture`.
- [ ] Empty state renders with "Add line" button for capture-permitted users.
- [ ] Add draft line with CPT `99213`, 1 unit, $125 → appears, status `draft`, total $125.
- [ ] Edit draft line unit price to $130 → row updates; no audit event yet.
- [ ] Delete draft line → row disappears.
- [ ] Post all drafts with ICD-10 → lines flip to `posted`, `charge.created` events emit.
- [ ] Post refuses when any line lacks ICD-10 → no lines transition; banner names the line.
- [ ] Void posted line with reason ≥ 5 chars → status `voided`, audit emits.
- [ ] Void refused for line on submitted claim → 409 banner.
- [ ] `ready_for_claim` banner + link appear once all lines posted.
- [ ] Impersonated user cannot post.
- [ ] Monorepo typecheck, vitest, build all pass; no `"use client"` introduced.

---

8. Not V1

- **AI code-suggestion import** — the add-form will gain a `[Pick from AI suggestions]` mode once the orchestrator lands (reads from `ai_scribe_code_suggestions` where `accepted_at IS NOT NULL`).
- **Fee-schedule lookup** — unit prices are hand-entered; fee-schedule autosuggest is a separate table + UI.

- **CPT master / search** — V1 uses free-text CPT input with a regex; typeahead against a CPT master is V2.
- **Bulk actions across encounters** — one encounter at a time.
- **Line-level approval workflow** — post-all transaction is all-or-nothing in V1.